LAPORAN PRAKTIKUM SEDERHANA TRANSACTION PROCEDURE

Nama : Zahra Zaskia Aulia Putri **Nim : 0110124011**

Kampus : sekolah tinggi teknologi terpadu nurul fikri **Semester : 5**

Aktifitas : Fullstack Web Developer **Tugas : Database mysql**

Tangga : 20/10/26

Link General Github : https://github.com/jaraa4/Tugas_SIB4_FWD.git

Link Tugas Github :

https://github.com/jaraa4/Tugas_SIB4_FWD/tree/main/Database/database%205/toko_db

---

Laporan ini menyajikan implementasi praktis dari dua konsep penting dalam manajemen basis data relasional: Transaction Management (COMMIT, ROLLBACK, SAVEPOINT) untuk menjamin integritas data (ACID) dan Stored Procedure untuk enkapsulasi logika bisnis tingkat basis data.

Seluruh pengujian dijalankan pada skema basis data perbankan dan ritel sederhana dengan tabel utama berikut:

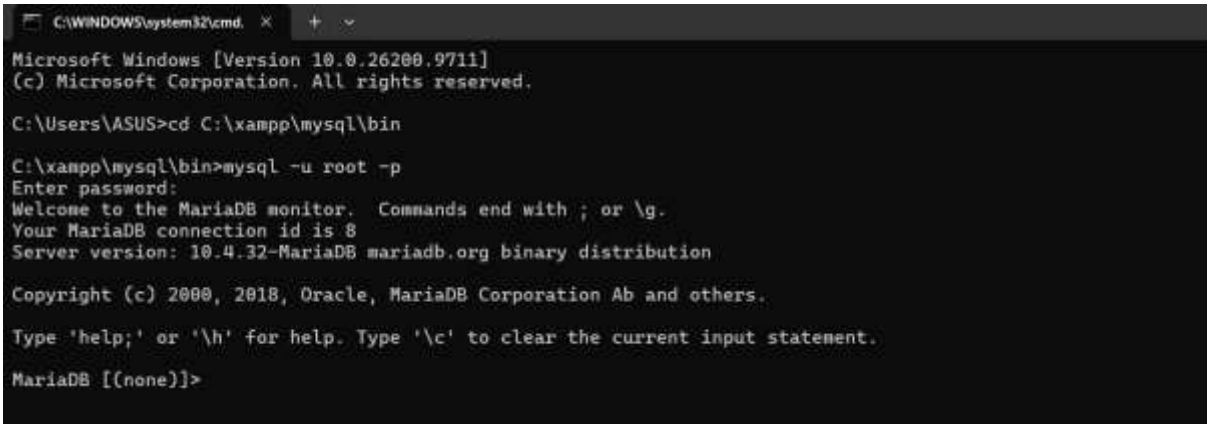
Nama Tabel	Kolom Utama	Deskripsi Fungsi
Rekening	no_rekening (PK), nama_nasabah, saldo	Menyimpan informasi akun dan saldo nasabah.
Produk	id_produk (PK), nama_produk, stok, harga	Menyimpan stok dan harga barang ritel.
Pesanan	id_pesanan (PK), id_produk (FK), jumlah, total_harga	Menyimpan transaksi pembelian produk.

1. Implementasi Transaction Management

Kasus 1: Transfer Dana Antar Rekening (Rollback saat Kegagalan)

Skenario: Transfer dana sebesar Rp 500.000 dari Rekening A ke Rekening B. Jika saldo Rekening A tidak mencukupi, seluruh rangkaian perintah harus dibatalkan (ROLLBACK) agar saldo tidak terpotong secara parsial.

a. Tekan tombol Windows + R pada keyboard Anda.
Ketik cmd lalu tekan Enter untuk membuka Command Prompt.
Masuk ke direktori MariaDB (asumsi menggunakan XAMPP di Windows). Ketik perintah berikut: cd C:\xampp\mysql\bin tekan enter, Hubungkan ke server MariaDB sebagai *root*: mysql -u root -p (Tekan Enter. Jika muncul permintaan password, kosongkan saja jika belum diatur, lalu tekan Enter kembali).



b. **Membuat Database dan Tabel Uji Coba**

Sebelum masuk ke Transaction dan Procedure, buatlah sebuah database latihan bernama toko_db beserta tabel rekening dan produk.

```
MariaDB [(none)]> CREATE DATABASE toko_db;
Query OK, 1 row affected (0.004 sec)

MariaDB [(none)]> USE toko_db
Database changed
MariaDB [toko_db]>
```

c. **Membuat tabel rekening (untuk kasus Transaction & Procedure)**

```
MariaDB [toko_db]> CREATE TABLE rekening (
    ->     id_rekening INT PRIMARY KEY AUTO_INCREMENT,
    ->     nama_pemilik VARCHAR(50),
    ->     saldo DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.017 sec)
```

d. **Mengisi data awal rekening**

```
MariaDB [toko_db]> INSERT INTO rekening (nama_pemilik, saldo) VALUES ('Budi', 500000), ('Ani', 300000);
Query OK, 2 rows affected (0.048 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

e. **Membuat tabel produk (untuk kasus Procedure & Transaction stok)**

```
MariaDB [toko_db]> CREATE TABLE produk (
    ->     id_produk INT PRIMARY KEY AUTO_INCREMENT,
    ->     nama_produk VARCHAR(50),
    ->     stok INT,
    ->     harga DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.015 sec)
```

f. **Mengisi data awal produk**

```
MariaDB [toko_db]> INSERT INTO produk (nama_produk, stok, harga) VALUES ('Laptop', 10, 7000000), ('Mouse', 25, 150000);
Query OK, 2 rows affected (0.009 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

2. **Implementasi Transaction Kasus Transaction**

   Kasus 1: Transfer Saldo Antar Rekening (Full Rollback jika Gagal)

   Skenario: Budi ingin mentransfer uang sebesar Rp100.000 ke Ani. Jika salah satu query gagal, saldo harus kembali seperti semula.

   a. Mulai transaction/transaksi

```
MariaDB [toko_db]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)
```

   b. Kurangi saldo Budi

```
MariaDB [toko_db]> UPDATE rekening SET saldo = saldo - 100000 WHERE nama_pemilik = 'Budi';
Query OK, 1 row affected (0.012 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

   c. Tambah saldo Ani

```
MariaDB [toko_db]> UPDATE rekening SET saldo = saldo + 100000 WHERE nama_pemilik = 'Ani';
Query OK, 1 row affected (0.000 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

   d. Periksa hasil sementara sebelum disimpan

```
MariaDB [toko_db]> SELECT * FROM rekening;
+--------------+---------------+------------+
| id_rekening | nama_pemilik | saldo      |
+--------------+---------------+------------+
|           1 | Budi          | 400000.00 |
|           2 | Ani           | 400000.00 |
+--------------+---------------+------------+
2 rows in set (0.003 sec)
```

   e. Jika saldo aman, simpan permanen:

```
MariaDB [toko_db]> COMMIT;
Query OK, 0 rows affected (0.007 sec)
```

3. **Kasus 2: Transaksi dengan `SAVEPOINT` (Rollback Parsial)**

   Skenario: Melakukan beberapa operasi pembelian, namun kita ingin membatalkan sebagian operasi saja tanpa membatalkan keseluruhan transaksi.

   a. Operasi 1: Beli produk pertama

```
MariaDB [toko_db]> INSERT INTO produk (nama_produk, stok, harga) VALUES ('Keyboard', 15, 300000);
Query OK, 1 row affected (0.001 sec)
```

   b. Buat titik simpan (Savepoint) di sini

```
MariaDB [toko_db]> SAVEPOINT sp_setelah_keyboard;
Query OK, 0 rows affected (0.000 sec)
```

   c. Operasi 2: Beli produk kedua (misalkan terjadi kesalahan/batal)

```
MariaDB [toko_db]> INSERT INTO produk (nama_produk, stok, harga) VALUES ('Monitor', -5, 1500000);
Query OK, 1 row affected (0.006 sec)
```

   d. Batalkan hanya sampai ke titik savepoint (membatalkan pembelian Monitor saja)

```
MariaDB [toko_db]> ROLLBACK TO sp_setelah_keyboard;
Query OK, 0 rows affected (0.002 sec)
```

   e. Simpan perubahan Keyboard secara permanen

```
MariaDB [toko_db]> COMMIT;
Query OK, 0 rows affected (0.007 sec)
```

f. Cek isi tabel produk

```
MariaDB [toko_db]> SELECT * FROM produk;
+------------+--------------+-------+-------------+
| id_produk | nama_produk | stok | harga       |
+------------+--------------+-------+-------------+
|          1 | Laptop       |    10 | 7000000.00 |
|          2 | Mouse        |    25 |  150000.00 |
|          3 | Keyboard     |    15 |  300000.00 |
+------------+--------------+-------+-------------+
3 rows in set (0.001 sec)
```

4. Kasus 1: Procedure dengan Parameter IN (Update Stok Produk)

Prosedur ini digunakan untuk memperbarui jumlah stok suatu produk berdasarkan ID-nya.

a. Ubah delimiter

```
MariaDB [toko_db]> DELIMITER //
MariaDB [toko_db]>
MariaDB [toko_db]> CREATE PROCEDURE UpdateStokProduk(IN p_id INT, IN p_tambah_stok INT)
    -> BEGIN
    ->     UPDATE produk
    ->     SET stok = stok + p_tambah_stok
    ->     WHERE id_produk = p_id;
    -> END //
Query OK, 0 rows affected (0.022 sec)

MariaDB [toko_db]>
MariaDB [toko_db]> DELIMITER ;
MariaDB [toko_db]>
```

b. Menambah 5 stok pada produk dengan ID 1 (Laptop)

```
MariaDB [toko_db]> CALL UpdateStokProduk(1, 5);
Query OK, 1 row affected (0.011 sec)
```

c. Cek hasilnya

```
MariaDB [toko_db]> SELECT * FROM produk WHERE id_produk = 1;
+------------+--------------+-------+-------------+
| id_produk | nama_produk | stok | harga       |
+------------+--------------+-------+-------------+
|          1 | Laptop       |    15 | 7000000.00 |
+------------+--------------+-------+-------------+
1 row in set (0.028 sec)
```

d. Kasus 2: Procedure dengan Parameter OUT (Menghitung Total Aset Saldo Nasabah) **Prosedur ini menghitung total keseluruhan saldo dari seluruh nasabah yang ada di tabel rekening dan mengembalikannya melalui variabel *output*. Ubah delimiter**

```
MariaDB [toko_db]> CREATE PROCEDURE HitungTotalSaldo(OUT total_semua DECIMAL(10,2))
    -> BEGIN
    ->     SELECT SUM(saldo) INTO total_semua FROM rekening;
    -> END //
Query OK, 0 rows affected (0.011 sec)

MariaDB [toko_db]>
MariaDB [toko_db]> DELIMITER ;
MariaDB [toko_db]>
```

e. Panggil procedure dengan memasukkan variabel output (@hasil)

```
MariaDB [toko_db]> CALL HitungTotalSaldo(@hasil);
Query OK, 1 row affected (0.003 sec)
```

f. Tampilkan nilai dari variabel output

```
MariaDB [toko_db]> SELECT @hasil AS Total_Seluruh_Saldo;
+----------------------+
| Total_Seluruh_Saldo |
+----------------------+
|           800000.00 |
+----------------------+
1 row in set (0.006 sec)
```